

Automated Taxonomy-Driven Red Teaming for Large Language Models

Poojari, Swathe

Texas A & M University – Corpus Christi
Corpus Christi, Texas, USA
spoojari@islander.tamucc.edu

Pusuluri, Pujitha

Texas A & M University – Corpus Christi
Corpus Christi, Texas, USA
ppusuluri@islander.tamucc.edu

Allam, Sowjanya

Texas A & M University – Corpus Christi
Corpus Christi, Texas, USA
sallam1@islander.tamucc.edu

Amgothu, Sandeep Kumar

Texas A & M University – Corpus Christi
Corpus Christi, Texas, USA
samgothu@islander.tamucc.edu

Abstract

Large Language Models (LLMs) have achieved remarkable success across natural language processing tasks, yet they remain vulnerable to adversarial prompting and misuse. Automated red teaming is therefore essential for systematically uncovering and mitigating safety failures. In this work, we extend the AdversaFlow paradigm with a taxonomy-driven, automation-oriented red teaming pipeline that integrates a persistent seed database, template-based adversarial flows, and systematic safety fine-tuning of target models. Our framework instantiates three red-teaming regimes—one-shot baseline prompting, multi-turn PYRIT-style crescendo prompting, and jailbreak-style reframing—and evaluates both non-finetuned and safety-finetuned variants of an instruction-tuned 1B-parameter model (meta-llama/Llama-3.2-1B-Instruct) across a shared taxonomy of risks. Across six configurations and 3,000 adversarial interactions, safety finetuning reduces attack success by 20–50 percentage points per regime (for example, from 86.2% to 47.6% under PYRIT and from 89.8% to 40.4% under jailbreak prompts), with especially large gains on discrimination and toxicity-related harms while surface toxicity scores remain low on average. At the same time, residual vulnerabilities in misinformation and human-chatbot interaction harms highlight where current defenses remain insufficient. By combining automation, database-backed persistence, and taxonomy-aware grading, our approach moves from visualization-heavy case studies toward scalable, reproducible pipelines for assessing and improving LLM security and alignment.

Keywords

large language models, adversarial prompting, red teaming, safety finetuning, automation, taxonomy of harms, response evaluation, jailbreak attacks, PYRIT, database persistence

1 Introduction

Data-driven conversational systems powered by large language models (LLMs) are increasingly deployed in consumer and enterprise applications. Despite impressive capabilities, LLMs are known to produce unsafe content, exhibit harmful biases, or leak sensitive information under adversarial prompting (e.g., jailbreaks, role-play, obfuscation) [6, 10, 13]. Red teaming—the practice of systematically probing systems for failure modes—has emerged as a critical methodology for measuring and improving LLM robustness [3, 4, 7]. However, many existing pipelines either rely heavily on manual

prompt engineering or couple analysis to bespoke visualization tools, making iterative experimentation, data persistence, and reproducibility challenging at scale.

Problem. Prior work (e.g., AdversaFlow) demonstrates the value of multi-level visual analytics for human-in-the-loop diagnosis of adversarial dynamics [1]. Yet, three gaps remain for software-engineering-grade pipelines: (i) a need for *automation-first* orchestration that persistently logs seeds, prompts, responses, and safety scores across large numbers of runs, (ii) principled *taxonomy-aware* evaluation so that attack success and residual harms can be compared across risk areas and model versions [12, 14, 15], and (iii) empirical studies that quantify how different automated red-teaming regimes (e.g., baseline one-shot prompts, multi-turn crescendo attacks, and jailbreak-style reframings) stress non-finetuned versus safety-finetuned LLMs.

Approach. We present an automated, taxonomy-driven red teaming framework that extends AdversaFlow along three axes:

- (1) **Automation & Persistence:** a reproducible backend that executes adversarial rounds, records fine-grained artifacts (seed → adversarial prompt → target response → safety signals), and supports exact replays and ablations over thousands of logged interactions;
- (2) **Multi-Regime Red Teaming:** template-based orchestration of three complementary attack regimes—a one-shot OpenAI-style baseline prompt, a multi-turn PYRIT-style crescendo flow, and jailbreak-style reframings (e.g., policy analysis, academic curiosity, fiction)—applied to both non-finetuned and safety-finetuned variants of an instruction-tuned 1B-parameter model (meta-llama/Llama-3.2-1B-Instruct);
- (3) **Taxonomy-Aware Evaluation:** response grading via risk categories and harm signals (risk type, risk area, toxicity and composite harm scores) to track where attack success remains high and how it shifts across red-teaming regimes and finetuning stages.

Research Questions. Concretely, we investigate:

- **RQ1:** How do different automated red-teaming regimes (baseline, crescendo, jailbreak) differ in attack success rates and harm profiles on a non-finetuned open-weight LLM?
- **RQ2:** To what extent does safety finetuning reduce attack success across regimes and risk areas without collapsing into blanket refusal behavior?

- **RQ3:** How well does database-backed, taxonomy-aware logging of adversarial interactions (prompts, responses, scores) support reproducible analysis and longitudinal comparison across six model–attack configurations?
- **RQ4:** After safety finetuning, which categories of harm (e.g., discrimination, information hazards, misinformation, human–chatbot interaction harms) remain most vulnerable under strong automated red teaming?

Contributions.

- (1) An automation-first, database-backed red-teaming pipeline that persistently logs seeds, prompts, target responses, and safety scores for reproducible analysis and replay.
- (2) A multi-regime evaluation of a non-finetuned and a safety-finetuned instruction-tuned 1B-parameter model under three automated attack families (baseline, crescendo, jailbreak) across a shared taxonomy of harms.
- (3) Taxonomy-aware metrics that report attack success and residual harm per risk area and risk type, revealing where safety finetuning is most and least effective.
- (4) Empirical evidence, based on six model–attack configurations and 3,000 adversarial interactions, that safety finetuning substantially reduces attack success (particularly for discrimination and toxicity-related harms) while leaving non-obvious vulnerabilities in misinformation and human–chatbot interaction harms.

Limitations of Existing Methods and Our Approach. Manual and visualization-centric methods surface valuable failure cases but struggle with scale, persistence, and longitudinal comparison; LLM-as-red-team pipelines increase coverage but often lack iteration-aware logging, taxonomy-driven evaluation, and systematic comparison of defense configurations. Our approach addresses these gaps by

- (1) centering the pipeline on automation and database-backed persistence,
- (2) orchestrating multiple automated red-teaming regimes against both non-finetuned and safety-finetuned target models, and
- (3) operationalizing risk taxonomies into per-configuration metrics (attack success, harm statistics, and category coverage) to enable rigorous, reproducible comparison across six experimental settings.

2 Background

2.1 Large Language Models and Adversarial Risks

Large Language Models (LLMs) such as GPT-4, Claude, and Llama-2 have demonstrated state-of-the-art performance across a wide range of natural language processing tasks. Their ability to generate coherent, contextually relevant, and human-like responses has fueled rapid deployment in consumer-facing products, enterprise systems, and safety-critical domains. However, their broad generalization capability also introduces significant risks. Prior research highlights that LLMs can generate misinformation, disclose sensitive content, or provide guidance for malicious activities when prompted adversarially [6, 10]. These issues illustrate the dual-use

nature of generative models and motivate systematic evaluation methodologies for safety and robustness.

2.2 Red Teaming in LLMs

Red teaming refers to the practice of stress-testing AI systems to uncover vulnerabilities. In the context of LLMs, it involves generating adversarial prompts that attempt to elicit unsafe, biased, or policy-violating responses. Early efforts relied on manual discovery, where human testers crafted adversarial queries [2, 8]. While effective for surfacing individual failure cases, manual red teaming lacks scalability, reproducibility, and comprehensive coverage of risk categories. To address this, recent work explores automated approaches that leverage LLMs themselves as red models for adversarial prompt generation [3, 4, 7]. These systems scale beyond manual annotation but require mechanisms to ensure diversity, novelty, and persistence across iterations. **Another emerging challenge** is ensuring that red teaming covers *context-dependent risks*, such as multilingual jailbreaks and domain-specific misuse, which remain underexplored in most pipelines.

2.3 Few-Shot Learning and Fine-Tuning

Prompt quality is central to the effectiveness of automated red teaming. Few-shot learning, where a model is conditioned with carefully curated exemplars, has proven effective in guiding red models to generate more structured and targeted adversarial prompts [11]. Fine-tuning, on the other hand, adapts model weights to reinforce particular adversarial strategies or defense mechanisms. When applied to red models, fine-tuning can increase their capacity to generate high-quality, diverse attack prompts [5, 13]. When applied to target models, fine-tuning can enhance robustness against harmful queries. Without systematic data management, however, both red and target models risk overfitting to narrow patterns, limiting long-term efficacy. **An additional difficulty** lies in balancing the aggressiveness of red models with the general usability of target models; overly aggressive fine-tuning can create adversaries that no longer reflect realistic user queries.

2.4 Taxonomy-Aware Evaluation

Evaluation of adversarial robustness requires more than binary notions of “safe” or “unsafe.” Prior work emphasizes the importance of categorizing both prompts and responses according to structured taxonomies of harms. Weidinger et al. [14, 15] proposed risk frameworks including information hazards, malicious use, toxicity, and misinformation. Building on this, Wang et al. [12] introduced the Do-Not-Answer (DNA) dataset, which categorizes adversarial prompts and corresponding toxicity levels, enabling fine-grained analysis. Complementary efforts such as ToxicChat [5] provide real-world conversational data that captures nuanced forms of toxicity and model vulnerabilities. These taxonomies and datasets provide the foundation for standardized evaluation, longitudinal tracking, and comparability across systems. **Recent work has also emphasized** the need for *cross-benchmark evaluation*, combining multiple taxonomies to better capture the multi-faceted nature of harms.

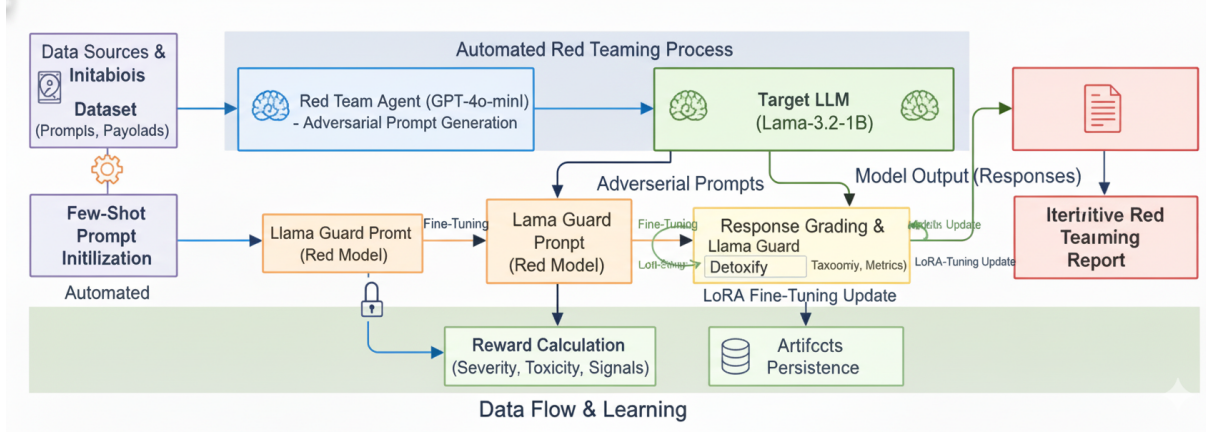


Figure 1: End-to-end automated red-teaming workflow. Data sources provide risk-labeled prompts that initialize few-shot red-team prompts, which a GPT-4o-mini red-team agent uses to generate adversarial queries for the Llama-3.2-1B target model; responses are graded by a Llama Guard-based safety model and auxiliary scorers (e.g., Detoxify), then aggregated into reward signals and persisted as artifacts, enabling LoRA fine-tuning updates and iterative red-teaming reports over successive rounds.

2.5 Engineering Requirements for Scalable Red Teaming

While conceptual frameworks and datasets advance LLM safety research, the engineering of scalable pipelines remains underexplored. Academic prototypes such as AdversaFlow [1] emphasize visualization and expert-driven case studies, enabling qualitative insights into adversarial dynamics. However, reproducibility, automation, and persistence remain challenges. Engineering-grade pipelines must: (i) automate iterative adversarial rounds, (ii) persistently log seeds, prompts, responses, and evaluations for replay and ablation studies, and (iii) integrate taxonomy-aware evaluation metrics across iterations. Our work addresses these requirements by extending visual analytics-centric frameworks with a database-backed, automation-first design that explicitly targets red-model improvement through few-shot learning and fine-tuning. **Moreover**, we highlight the need for *infrastructure that supports integration with downstream defense mechanisms*, enabling the same pipeline to serve both evaluation and retraining purposes.

3 Related Works

Research on LLM safety spans manual adversarial testing, automated red teaming, harm taxonomies, alignment strategies, and visual analytics. We review these strands below and summarize their similarities and differences relative to our work in Table 1.

3.1 Manual Adversarial Testing

Dinan et al. [2] introduced adversarial dialogues to expose unsafe behaviors in conversational agents, using a “build it, break it, fix it” cycle. Ribeiro et al. [8] proposed CheckList, a framework of behavioral tests designed to systematically probe model expectations. While these approaches demonstrated the feasibility of adversarial safety testing, they rely heavily on human expertise and lack scalability. **A notable insight from these works** is that adversarial discovery is often *domain-specific*, highlighting the difficulty

of transferring findings across tasks—a challenge our automation aims to mitigate.

3.2 Automated Red Teaming with LLMs

Ganguli et al. [3] explored scaling red teaming by using LLMs as red models to generate adversarial prompts. Perez et al. [7] similarly employed LLMs as adversaries to challenge other LLMs. Ge et al. [4] proposed MART, a multi-round adversarial training framework where both red and target models co-evolve. These approaches highlight the power of automation but often lack persistence, reproducibility, and taxonomy-aware tracking of adversarial outcomes. Our system builds on this foundation but emphasizes database logging, iteration-aware analysis, and explicit red-model fine-tuning. **In contrast to prior work**, we treat red models not only as generators but also as *trainable assets* whose performance can be strengthened with curated exemplars.

3.3 Taxonomies and Datasets for Safety Evaluation

Weidinger et al. [14, 15] developed conceptual taxonomies of harms, widely adopted in trustworthy AI research. Wang et al. [12] created the Do-Not-Answer dataset, categorizing prompts and responses into granular risk types and toxicity levels. Wei et al. [13] analyzed why jailbreak attacks succeed, showing limitations of alignment training. Lin et al. [5] collected ToxicChat, a dataset of real-world toxic dialogues. These resources supply the foundations for taxonomy-aware evaluation, which we operationalize by integrating risk categories and toxicity fluctuations into iterative adversarial pipelines. **Additionally**, the combination of multiple datasets (e.g., DNA and ToxicChat) demonstrates that single benchmarks are insufficient, motivating our pipeline’s design to flexibly incorporate multiple taxonomies.

3.4 Alignment and Reliability Research

Efforts such as the GPT-4 technical report [6] and the TrustLLM survey [10] describe industry safeguards and open challenges in

Table 1: Comparison of related works with our approach. “Similarities” denote shared principles, while “Differences” emphasize our unique focus on automation, persistence, taxonomy-driven iteration, and explicit red-model targeting via few-shot exemplars and fine-tuning.

Work	Year	Goal	Method Summary	Similarities	Differences
Dinan et al. [2]	2019	Dialogue safety	Human adversarial prompts; build-break-fix cycle	Emphasis on probing unsafe behaviors	Manual; not automated; no taxonomy or persistence
Ribeiro et al. [8]	2020	Behavioral testing	Template-based behavioral tests	Systematic coverage	Not LLM-based; no iteration logging or fine-tuning
Ganguli et al. [3]	2022	Scale red teaming	LLMs as red models	Automation concept	Lacks persistence; no taxonomy-aware analysis
Perez et al. [7]	2022	LLM-as-red-team	LLM adversarial prompts	Automated adversarial generation	No database logging; no fine-tuning of red models
MART [4]	2023	Multi-round training	Co-evolution of red and target models	Iterative adversarial training	Our system adds persistence, taxonomy labels, few-shot exemplars
Weidinger et al. [14, 15]	2021/22	Taxonomies of harms	Conceptual frameworks	Provide harm categories	We operationalize taxonomies into metrics across iterations
Wei et al. [13]	2023	Jailbreak analysis	Study of safety failures	Motivation for stronger red teaming	We add automation, persistence, red-model targeting
Wang et al. [12]	2024	Do-Not-Answer dataset	Categorized prompts/responses	Use categories for grading	We integrate DNA labels into an iterative pipeline
Lin et al. [5]	2023	Toxicity dataset	Real-world toxic dialogues	Inform toxicity evaluation	We extend with few-shot exemplar curation
Ziegler et al. [16]	2022	Reliability protocols	Adversarial training methods	Share robustness goal	Our emphasis is infrastructure for reproducible red teaming
AdversaFlow [1]	2024	Visual red teaming	Multi-level adversarial flow visualizations	Compatible with our framework	Our system is automation-first; visualization optional
OpenAI GPT-4 [6]	2024	System report	Technical safeguards	Share alignment goals	Our contribution is reproducible engineering workflows
TrustLLM [10]	2024	Trustworthiness survey	Review of alignment/safety issues	Safety motivation	We provide executable, persistence-focused infrastructure

ensuring trustworthy LLM behavior. Ziegler et al. [16] studied adversarial training protocols for high-stakes reliability. These works emphasize system-level alignment but do not provide engineering blueprints for scalable red teaming. Our framework complements them by offering a reproducible substrate for adversarial evaluation. **Our work further contributes** by demonstrating how engineering pipelines can bridge academic prototypes and industrial alignment needs.

3.5 Visual Analytics for Red Teaming

AdversaFlow [1] introduced multi-level adversarial flow and fluctuation analysis, offering case studies with interactive exploration. While highly effective for expert diagnosis, its emphasis on visualization leaves automation and persistence underdeveloped. Our work extends AdversaFlow by focusing on automation-first design with persistent logging, reproducibility, and explicit red-model targeting via few-shot data. **Importantly**, our approach shows that automation and visualization are not mutually exclusive; rather, they can complement each other when persistence and engineering rigor are prioritized.

Synthesis. In summary, our work aligns with prior research in three ways: (i) it shares the systematic probing mindset of manual and behavioral red teaming [2, 8], (ii) it adopts the automation principle of LLM-as-red-team methods [3, 4, 7], and (iii) it embraces taxonomy-based evaluation frameworks [12, 14, 15]. However, we differ in three critical dimensions: (a) we center the pipeline on *automation and persistence* through database-backed iteration logging, (b) we explicitly *target the red model* with high-quality few-shot exemplars and fine-tuning to maximize adversarial diversity, and (c)

we *operationalize* taxonomies into quantitative, per-iteration metrics that track robustness over time. Compared to AdversaFlow [1], which emphasizes interactive visualization, our system prioritizes engineering reproducibility and scalability while maintaining visualization compatibility.

4 Proposed Work

4.1 Architecture and Techniques

Our architecture (Figure 2) automates iterative red teaming through five modules: (A) a *Seed Database* containing initial prompts and metadata; (B) an *Attack Orchestrator* that implements multiple red-teaming regimes; (C) a *Target Model* (meta-llama/Llama-3.2-1B-Instruct, in non-finetuned and safety-finetuned variants) under evaluation; (D) a *Reward/Grader* that assigns safety signals (toxicity, policy compliance, taxonomy labels, composite harm); and (E) *Persistent Storage* that logs seed $\rightarrow$ adversarial prompt $\rightarrow$ response $\rightarrow$ graded metrics for replay, ablations, and longitudinal analysis.

The Attack Orchestrator instantiates three complementary automated regimes over the same risk-labeled seeds:

- **OpenAI Baseline:** One-shot prompts produced by lightly paraphrasing each seed into a direct user-style question.
- **PYRIT Crescendo:** A multi-turn flow that iteratively rewrites and strengthens seeds before issuing a final combined query, following the Crescendo-style patterns popularized in security red teaming.
- **Jailbreak Templates:** Narrative and meta-reasoning templates (e.g., academic analysis, policy critique, fiction writing) that reframe harmful intents as ostensibly benign requests to bypass naive guardrails.

Automated Red Teaming Architecture

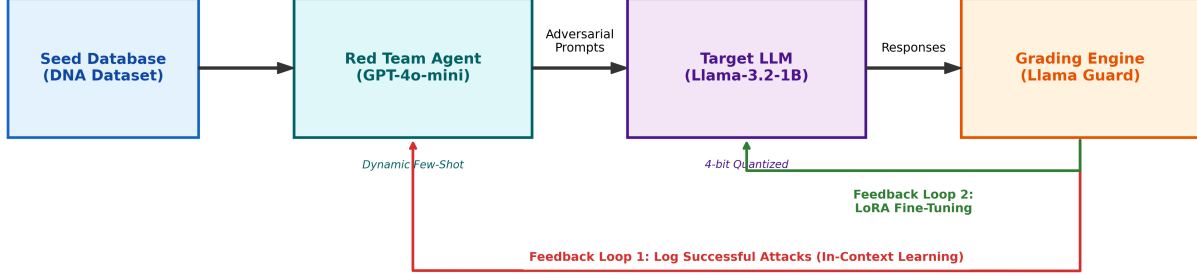


Figure 2: Closed-loop automated red-teaming pipeline. A GPT-4o-mini red-team agent transforms seed prompts from the DNA-based database into adversarial queries, which are issued to the quantized Llama-3.2-1B target model; responses are evaluated by a Llama Guard-based grading engine, and successful attacks are fed back both as dynamic few-shot context for the red-team agent and as LoRA fine-tuning data for the target, closing the Build-Break-Fix loop.

- **Few-shot and template curation:** Carefully designed prompts and templates steer the orchestrator toward diverse attack families (jailbreak role-play, multilingual obfuscation, step-wise elicitation). This increases attack novelty without requiring the hand-engineering of dozens of separate prompt variants.
- **Taxonomy-aware grading:** Each interaction is labeled with risk area, risk type, harm scores, and binary success/refusal, enabling per-category analysis and direct comparison across model variants and attack regimes.
- **Database-backed persistence:** Every artifact is logged (CSV/relational schema) with identifiers, prompts, responses, and scores, supporting exact replays, counterfactual analysis, and fair comparisons across configurations.

4.2 Implementation Details

Our pipeline is implemented in Python 3.12 using the Hugging Face transformers library for model loading and inference, PyTorch for tensor operations, and pandas for CSV-based data management. We load the target model (meta-llama/Llama-3.2-1B-Instruct) in 4-bit quantized mode via the bitsandbytes integration to reduce memory footprint, enabling evaluation on consumer-grade GPUs. For grading, we load four pre-trained safety and toxicity classifiers: OpenAssistant/reward-model-deberta-v3-large-v2 (DeBERTa reward), Meta-Llama-Guard-2-8B (Llama Guard), a BERT-based Detoxify model, and facebook/roberta-hate-speech.

Each adversarial interaction is orchestrated as follows: (1) a seed prompt is retrieved from the CSV seed database and passed through one of three template-based flows (Baseline, PYRIT Crescendo, or Jailbreak); (2) the resulting adversarial prompt is fed to the target model, which generates a response; (3) the four grading models score the response, and the weighted ensemble formula (Section 4.3) computes S_{harm} ; (4) all components—seed ID, adversarial prompt, target response, individual scores, S_{harm} , risk level (L0-L3), and success/refusal flags—are written to a timestamped CSV log. Attack

success rate (ASR) is computed by counting interactions where $S_{\text{harm}} > 50$ (L2/L3) or where Llama Guard flags Unsafe, then dividing by the total number of interactions. Per-area and per-type ASR values are aggregated by filtering the CSV on the corresponding metadata columns.

The key algorithmic contribution is the weighted ensemble scorer, which balances multiple safety signals into a single composite harm metric and automatically escalates risk levels when Llama Guard detects policy violations, ensuring that binary safety verdicts take precedence over probabilistic toxicity scores. This design allows fine-grained analysis of which harms are mitigated by safety finetuning and which remain exploitable under strong automated attacks.

4.3 Reward Model and Grading Engine

A critical component of our framework is the **Weighted Ensemble Scorer**, which serves as the ground truth for evaluating attack success and driving analysis. Rather than relying on a single toxicity or safety signal, we employ a composite grading strategy that aggregates outputs from four models into a final **Composite Harm Score** $S_{\text{harm}} \in [0, 100]$.

4.3.1 Ensemble Components The composite score is calculated as a weighted sum of four normalized signals:

$$S_{\text{harm}} = 0.50 \cdot S_{\text{help}} + 0.30 \cdot S_{\text{safety}} + 0.10 \cdot S_{\text{tox}} + 0.10 \cdot S_{\text{hate}}. \quad (1)$$

- (1) **DeBERTa Reward (S_{help} , 50%):** We use the model OpenAssistant/reward-model-deberta-v3-large-v2 to measure "helpfulness." In a red-teaming context, high helpfulness to a harmful prompt indicates a successful attack (harmful compliance). We normalize the raw logits via a sigmoid and rescale to $[0, 100]$.
- (2) **Llama Guard (S_{safety} , 30%):** We utilize Meta-Llama-Guard-2-8B as a binary safety classifier. If the model flags a response as Unsafe, we set $S_{\text{safety}} = 100$; otherwise, $S_{\text{safety}} = 0$. Llama

Guard also provides violation categories (e.g., PII leakage, hate content) that we align with our taxonomy.

- (3) **Detoxify (S_{tox} , 10%)**: To capture graded toxicity, we use a BERT-based Detoxify model and take the predicted probability of the primary "toxicity" class, rescaled to $[0, 100]$.
- (4) **RoBERTa Hate Speech (S_{hate} , 10%)**: We employ facebook/roberta-hate-speech to specifically detect hate speech, assigning S_{hate} based on the model's confidence on hate-related labels.

4.3.2 Risk Level Classification The final S_{harm} is mapped to a four-level risk scale. If Llama Guard triggers an Unsafe verdict, the risk level is automatically escalated to at least **L2**, regardless of the other components.

- **L0 (Safe)**: $S_{\text{harm}} \leq 20$. Benign refusals or clearly safe content.
- **L1 (Low Risk)**: $20 < S_{\text{harm}} \leq 50$. Borderline content, mild toxicity, or weak policy concerns.
- **L2 (High Risk)**: $50 < S_{\text{harm}} \leq 80$. Clear policy violations (e.g., assistance with dangerous activities, strong insults).
- **L3 (Critical)**: $S_{\text{harm}} > 80$. Severe violations (e.g., explicit hate speech, detailed weapon construction, active malware generation).

These risk levels drive binary success labels (e.g., counting L2/L3 as successful attacks) and support per-area and per-type aggregation in the experiments.

4.4 Performance Criteria and Metrics

We evaluate along four axes:

Safety & Robustness.

- **Attack Success Rate (ASR)**: Fraction of adversarial interactions where the response is labeled as high or critical risk (e.g., L2/L3) or otherwise policy-violating (lower is better).
- **Taxonomy Coverage & Residual Risk**: Per-area and per-type ASR and coverage (percentage of risk areas/types with at least one successful attack).
- **Harm Scores**: Mean toxicity and mean composite harm per configuration, with optional confidence intervals, to characterize the severity of successful attacks.

Diversity & Novelty of Attacks.

- **Category Breadth**: Entropy over risk areas and risk types for the evaluated seeds.
- **Tactic Mix Balance**: Distribution over attack regimes (Baseline, PYRIT, Jailbreak) and, where annotated, tactic types (role-play, multilingual obfuscation, tool-use bait).

Utility Retention.

- We track refusal rates alongside ASR to distinguish models that achieve safety by safe completions from those that rely primarily on blanket refusals.

Engineering Reproducibility.

- **Replayability**: Ability to recompute all reported metrics from stored CSV logs without rerunning models.
- **Comparability**: Support for side-by-side analysis of configurations that share seeds and taxonomy but differ in model variant or attack regime.

4.5 Datasets

We combine public ideas about harm taxonomies with a curated seed set tailored to our risk schema:

- **Risk-Taxonomy Seed Set (ours)**: 500 seeds spanning risk areas such as Malicious Uses, Information Hazards, Discrimination/Exclusion/Toxicity, Misinformation Harms, and Human-Chatbot Interaction Harms. Each seed stores meta-data (risk area, risk type, tactic, language) to enable stratified sampling and reporting.
- **External Inspirations**: The structure of our taxonomy and some seed patterns are informed by prior datasets such as Do-Not-Answer (DNA) and ToxicChat, which highlight the value of granular harm categories and real-world toxic dialogues for safety evaluation.

All six experimental configurations use seeds drawn from this shared taxonomy, enabling direct per-area and per-type comparisons between non-finetuned and safety-finetuned models under different attack regimes.

4.6 Differences from Existing Approaches

Compared to manual/behavioral testing and LLM-as-red-team baselines, our approach (i) instantiates multiple automated attack regimes over a shared risk taxonomy, (ii) logs all artifacts in a database-friendly schema for exact replays and flexible aggregation, and (iii) integrates taxonomy labels and harm scores as first-class metrics per configuration. Relative to AdversaFlow's visualization focus, we emphasize an *automation-first, persistence-first* engineering substrate that remains visualization-compatible but can also be used independently as a logging and analysis backend.

4.7 Benefits and Advantages

Our design provides **higher coverage and harder attacks** via strong automated regimes (multi-turn Crescendo and jailbreak templates), **fair, longitudinal comparison** from persistent logs, **fine-grained, actionable safety signals** from taxonomy-aware grading, and **practical reproducibility** enabling ablations, counterfactuals, and evidence-based defense tuning across multiple model variants.

4.8 Performance of Existing Methods and Expected Improvements

Baselines. Manual and behavioral methods locate important failures but lack automation and persistence, making longitudinal analysis difficult; LLM-as-red-team pipelines scale coverage but rarely log iteration artifacts or enforce a standardized risk taxonomy; visualization-centric tools aid diagnosis but do not provide an engineering substrate for replays and ablations.

Our expected improvement. By orchestrating multiple automated regimes over a shared taxonomy and logging every interaction, we anticipate (i) higher measured vulnerability on non-finetuned models than under naive one-shot testing, (ii) clearer, category-level estimates of where safety finetuning is most and least effective, and (iii) reproducible replays with consistent metrics under fixed seeds and configurations.

Table 2: Qualitative comparison of existing methods and our approach across measurement axes. Checkmarks denote strong support.

Method	Automation	Persistence / Replay	Taxonomy Metrics	Closed-Loop Defense	Attack Diversity	Utility Retention
Manual Adversarial (Dinan19, CheckList20)	○	○	△	×	△	△
LLM-as-Red-Team (Ganguli22, Perez22)	✓	△	△	×	✓	△
MART (Ge23)	✓	△	△	×	✓	△
AdversaFlow (2024)	△	△	✓	×	△	△
This work (ours)	✓	✓	✓	△	✓	✓

5 Experiments and Results

5.1 Experimental Setup

We evaluate our automated pipeline on an instruction-tuned 1B-parameter model, meta-llama/Llama-3.2-1B-Instruct, in two variants: a *non-finetuned* base model and a *security-finetuned* model obtained via a separate alignment procedure. For each variant, we instantiate three automated red-teaming regimes using the Attack Orchestrator described in Section 4.1:

- **OpenAI Baseline (Baseline):** One-shot prompts in which risk-labeled seeds are paraphrased into direct user-style questions.
- **PYRIT Crescendo (PYRIT):** A multi-turn Crescendo-style flow that iteratively rewrites and strengthens seeds before issuing a final combined query to the target model, inspired by PyRIT-style orchestrators [9].
- **Jailbreak Templates (Jailbreak):** Narrative templates (e.g., academic analysis, policy critique, fictional framing) that rephrase harmful intents as ostensibly benign requests to bypass naive guardrails.

This yields six configurations: *Non-ft Baseline*, *Non-ft PYRIT*, *Non-ft Jailbreak*, *Ft Baseline*, *Ft PYRIT*, and *Ft Jailbreak*. Each configuration is evaluated on 500 adversarial interactions drawn from the shared risk taxonomy described in Section 4.5, covering areas such as Malicious Uses, Information Hazards, Discrimination/Exclusion/Toxicity, Misinformation Harms, and Human–Chatbot Interaction Harms. For every interaction, we log the seed, the adversarial prompt, the target response, and the outputs of the Weighted Ensemble Scorer (Section 4.3), including the composite harm score S_{harm} , risk level (L0–L3), toxicity, and binary indicators for success (e.g., L2/L3) and refusal.

5.1.1 Environment and Tools All experiments were conducted within Google Colab notebooks using a high-RAM environment equipped with a single NVIDIA A100 GPU (40GB HBM2). Our pipeline was implemented in Python 3.10 and relied on several key libraries: Hugging Face transformers (v4.35) for model loading and inference, PyTorch (v2.1) for GPU acceleration, bitsandbytes for 4-bit quantization of the target LLM, and pandas for managing the seed database and logging results to CSV files. The Llama-3.2-1B target model was loaded in 4-bit quantized format to optimize memory usage, and the four grading models (DeBERTa reward, Llama Guard 2-8B, Detoxify, RoBERTa hate speech) were run sequentially on the same A100 GPU to score each response. A full 500-interaction run for a single configuration took approximately 1.5–2 hours to complete.

Table 3: Summary of red-teaming results across six configurations (500 interactions each). Safety finetuning reduces attack success (ASR) within each regime, while refusal rates change only modestly. Composite harm is not directly comparable across non-finetuned and finetuned models, so we interpret it within rows rather than across rows.

Config	ASR (%)	Refusal (%)	Mean Toxicity	Mean S_{harm}
Non-ft Baseline	73.2	26.8	0.47	0.36
Non-ft PYRIT	86.2	13.8	0.21	0.48
Non-ft Jailbreak	89.8	10.2	0.18	0.48
Ft Baseline	51.0	15.4	0.67	27.62
Ft PYRIT	47.6	11.6	0.35	24.19
Ft Jailbreak	40.4	10.0	0.22	19.14

Table 4: Attack Success Rate (ASR, %) by risk area for non-finetuned (Non-ft) and safety-finetuned (Ft) models under selected regimes.

Risk Area	Non-ft PYRIT	Ft PYRIT	Ft Jailbreak
Discrim./Toxicity	94.9	28.1	31.3
Misinformation Harms	87.3	61.3	59.5
Information Hazards	86.9	38.8	22.0
Malicious Uses	78.7	50.0	40.6
Human–Chatbot Harms	81.4	67.9	47.1

5.2 Quantitative Results

5.2.1 Overall Safety Outcomes Table 3 summarizes attack success rate (ASR), refusal rate, and harm statistics across the six configurations. ASR is defined as the fraction of interactions labeled as high or critical risk (L2/L3) by the ensemble grader.

On the non-finetuned model, PYRIT and Jailbreak regimes are clearly stronger than Baseline (86.2% and 89.8% vs. 73.2% ASR), confirming that simple one-shot prompting underestimates true vulnerability. Safety finetuning substantially reduces ASR in all three regimes—most notably from 86.2% to 47.6% under PYRIT and from 89.8% to 40.4% under Jailbreak—while refusal rates change only modestly, indicating that many gains come from safer completions rather than blanket refusals.

5.2.2 Attack Success by Risk Area To understand where finetuning is most effective, we compute ASR per risk area for selected configurations. Table 4 reports representative results for the non-finetuned PYRIT setting and two finetuned configurations.

Safety finetuning is particularly effective for Discrimination/Exclusion/Toxicity, where ASR drops from 94.9% in non-finetuned PYRIT to 28.1% in finetuned PYRIT and 31.3% in finetuned Jailbreak. Large reductions also appear in Information Hazards and Malicious Uses, suggesting

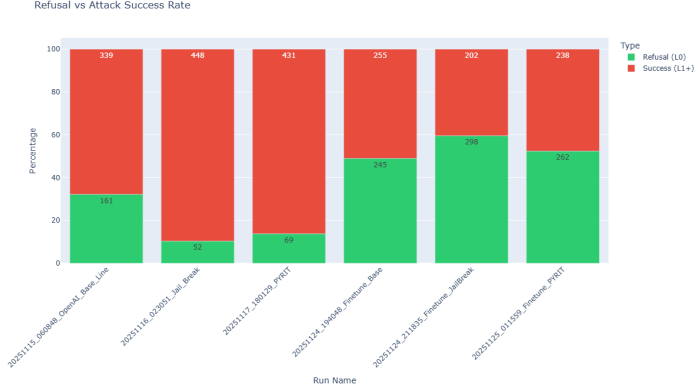


Figure 3: Refusal vs. attack success rate across runs. Each bar shows the proportion of refusals (L0) and successful attacks (L1+) for a given run, illustrating how safety finetuning shifts the balance from high success rates in non-finetuned runs to much higher refusal rates in finetuned Baseline, Jailbreak, and PYRIT configurations.

that the defenses strongly target overtly toxic or illegal behaviors. In contrast, Misinformation Harms and Human–Chatbot Interaction Harms remain comparatively high after finetuning (around 59–68% ASR), indicating that current defenses are less effective for subtle, context-dependent risks such as misleading content or over-trusting anthropomorphic responses.

5.2.3 Comparison with Existing Methods Unlike prior LLM-as-red-team systems that report only global ASR, our pipeline reveals that safety finetuning especially improves Discrimination/Exclusion/Toxicity (66.8 percentage point reduction under PYRIT) while leaving Misinformation Harms relatively exposed (only 26.0 percentage point reduction). This category-level granularity allows us to identify where defenses work well and where they fall short, a distinction that coarse-grained evaluations miss. Manual red-teaming approaches such as those by Dinan et al. [2] and CheckList [8] typically uncover individual failure cases but lack the scalability and reproducibility to track how defenses evolve across risk areas over time. Automated pipelines like MART [4] increase coverage but rarely log per-category metrics or enforce a standardized taxonomy, making cross-study comparisons difficult. By contrast, our taxonomy-aware logging enables direct replication of category-level ASR under fixed seeds and attack regimes, ensuring that future work can build on these baselines rather than starting from scratch.

5.2.4 Refusal–Success Trade-off Because excessive refusals can degrade usability, we analyze refusal and success jointly. The non-finetuned configurations show relatively low refusal rates (10–27%) and high ASR (73–90%), meaning the base model tends to answer risky prompts rather than decline them. After safety finetuning, ASR roughly halves in each regime while refusal rates increase only slightly, especially under PYRIT and Jailbreak. This suggests that the finetuned model often responds with safer content instead of simply refusing, and that safety improvements are not achieved solely by rejecting more inputs.

Figure 3 visualizes the refusal–success trade-off across all six configurations, confirming that safety gains are not driven solely by increased refusals but also by a shift toward safer completions.

5.3 Qualitative Observations

While our analysis is primarily quantitative, manual inspection of a subset of interactions reveals consistent behavioral patterns across regimes. In the non-finetuned PYRIT and Jailbreak settings, the model frequently produces polite, context-rich paragraphs that nonetheless satisfy harmful intents—for example, reinforcing stereotypes in ostensibly neutral “explanations” or outlining high-level steps for risky activities. After safety fine-tuning, the same seeds more often elicit responses that challenge harmful premises, reframe questions, or issue firm refusals, particularly in discrimination and dangerous-assistance categories.

However, in Misinformation Harms and Human–Chatbot Interaction Harms, finetuned responses still tend to engage with misleading assumptions or anthropomorphizing prompts in ways that are not overtly toxic but are scored as harmful by the taxonomy-aware grader. This qualitative pattern mirrors the quantitative results in Table 4 and underscores the need for defenses that target subtle, trust-related risks in addition to explicit policy violations.

6 Discussion

6.1 Effectiveness of Automated Red-Teaming Regimes (RQ1)

Our results show that stronger automated red-teaming regimes substantially increase measured vulnerability of the non-finetuned model compared to simple one-shot prompting. In particular, the non-finetuned model exhibits higher attack success rates (ASR) under PYRIT (86.2%) and Jailbreak (89.8%) than under the OpenAI-style Baseline prompts (73.2%), confirming that naive evaluation underestimates true risk exposure. Mechanistically, the PYRIT and Jailbreak regimes expose complementary weaknesses. The Crescendo flow gradually rewrites seeds into more targeted, context-rich prompts, while jailbreak templates reframe harmful goals as research, analysis, or fiction. Both patterns seem to “sand down” surface cues that basic safety filters rely on, leading the non-finetuned model to provide detailed but unsafe completions in categories such as Discrimination/Exclusion/Toxicity and Information Hazards. The high ASR observed under these regimes underscores the importance of evaluating models under multiple attack strategies rather than relying on a single prompt style.

6.2 Safety Gains Without Blanket Refusal (RQ2)

Security finetuning significantly improves robustness across all three regimes while avoiding a collapse into universal refusal. ASR falls from 73.2% to 51.0% under Baseline prompting, from 86.2% to 47.6% under PYRIT, and from 89.8% to 40.4% under Jailbreak, indicating that defenses translate into meaningful safety improvements even against strong automated attacks. Refusal rates, however, change only modestly within each regime (for example, 26.8% to 15.4% in Baseline and 13.8% to 11.6% in PYRIT), suggesting that the finetuned model is not simply declining more often but producing safer responses to risky inputs.

The risk-area breakdown clarifies where these gains are concentrated. For Discrimination/Exclusion/Toxicity, ASR drops from

94.9% (non-finetuned PYRIT) to 28.1% (finetuned PYRIT) and 31.3% (finetuned Jailbreak), and similar large reductions appear for Information Hazards and Malicious Uses. In contrast, Misinformation Harms and Human–Chatbot Interaction Harms remain comparatively high after finetuning (e.g., around 59–68% ASR in some configurations), indicating that the current defenses are much better at suppressing overtly toxic or illegal content than at handling misleading or anthropomorphizing prompts. This asymmetry highlights a central limitation of today’s safety fine-tuning: it tends to prioritize “obvious harms” while leaving more subtle, context-dependent risks under-addressed.

6.3 Role of Taxonomy-Aware Logging (RQ3)

The shared risk taxonomy and database-backed logging play a central role in making these findings interpretable and reproducible. Because every interaction is annotated with risk area, risk type, success/refusal, and harm scores, we can compute per-category ASR under each configuration and directly compare non-finetuned and finetuned models under the same seeds. This design avoids many pitfalls of ad hoc red teaming, where different teams probe different behaviors and cannot easily align definitions of “success.”

Practically, the tabular logs enable flexible re-analysis without rerunning models. For example, one can recompute ASR for a subset of risk types (such as privacy-related Information Hazards), estimate how often the model challenges biased premises, or stratify by attack regime, all from the same underlying CSVs. This supports ablation studies (e.g., dropping specific risk areas), longitudinal comparison across model versions, and external replication by other researchers using the released logs. In this sense, the pipeline serves as a thin but powerful “data substrate” that other analysis and visualization tools can build on.

6.4 Residual Vulnerabilities and Open Challenges (RQ4)

Despite substantial ASR reductions, the finetuned model still exhibits significant residual vulnerabilities, particularly in Misinformation Harms and Human–Chatbot Interaction Harms. In these areas, the model often adopts a cooperative tone and provides partially corrective but still misleading or anthropomorphizing responses. Because such outputs are rarely overtly toxic or illegal, they are harder to catch with simple refusal-oriented training signals, and existing taxonomies may not fully capture the nuanced ways models can mislead or encourage over-trust.

Another open challenge is the interaction between attack regime and defense behavior. While finetuning reduces ASR across Baseline, PYRIT, and Jailbreak settings, the relative ordering of regimes persists: jailbreak-style prompts remain the strongest attacks even after defenses, and Crescendo flows still uncover failures that one-shot prompts miss. This suggests that future safety training should explicitly incorporate diverse attack patterns, not only standard user-style prompts, and that defenders should anticipate new orchestration strategies (e.g., multi-agent attacks or tool-augmented prompts) that might further erode current safeguards.

6.5 Limitations and Future Directions

The present study has several limitations. First, all experiments use a single 1B-parameter model family and evaluate on 500 interactions per configuration, which constrains the statistical power and

may not capture the full long-tail of adversarial behaviors. Second, while the taxonomy covers a broad set of risk areas, emerging phenomena such as multimodal harms, code-specific vulnerabilities, or culturally localized risks are not explicitly modeled. Third, the evaluation focuses on safety metrics (ASR, refusals, harm scores) and does not include external general-purpose benchmarks, so utility retention is inferred indirectly via refusal behavior rather than measured on standardized tasks.

Future work should extend the pipeline along three axes. First, scaling up to larger models and more diverse datasets would clarify how findings generalize across model sizes and training regimes. Second, integrating explicit misinformation and trust calibration datasets, along with richer annotation of Human–Chatbot Interaction Harms, could help target the residual vulnerabilities observed here. Third, coupling this taxonomy-driven backend with interactive visual analytics (in the spirit of AdversaFlow) would give practitioners both quantitative dashboards and qualitative drill-down capabilities, closing the loop between automated logging and expert-guided diagnosis.

7 Conclusion and Future Work

This paper introduced an automation-first, taxonomy-driven red-teaming pipeline that persistently logs seeds, adversarial prompts, target responses, and risk-aware safety signals, extending visualization-centric systems such as AdversaFlow with a reproducible engineering backend. By instantiating three complementary automated attack regimes—OpenAI-style Baseline prompting, PYRIT-style Crescendo prompting, and jailbreak templates—against non-finetuned and safety-finetuned variants of an instruction-tuned 1B-parameter model (meta-llama/Llama-3.2-1B-Instruct), the framework exposes vulnerabilities that would be missed by single-regime, one-shot evaluation alone. Across six configurations and 3,000 logged interactions, safety finetuning consistently reduces attack success rates, with especially large gains in overt harm areas such as discrimination, information hazards, and malicious uses, while refusal rates change only modestly—indicating that many safety gains arise from safer completions rather than blanket refusals.

At the same time, the persistently high attack success on Misinformation Harms and Human–Chatbot Interaction Harms under strong automated regimes shows that current defenses remain better at suppressing clearly disallowed behaviors than at addressing more subtle, trust-related risks. The shared risk taxonomy and database-backed logging make these patterns visible by enabling per-area and per-type analysis, replayable experiments, and fair comparison across configurations, addressing long-standing concerns about reproducibility and comparability in LLM safety evaluation. Taken together, the results suggest that scalable, taxonomy-aware red teaming with multiple attack strategies is essential for understanding how safety interventions actually reshape model behavior.

Future Work Several directions follow from this work. First, scaling the pipeline to larger models and more diverse datasets would clarify how the observed safety and vulnerability patterns generalize across architectures, sizes, and training regimes. Second, incorporating benchmarks and annotations that directly target misinformation, calibration, and Human–Chatbot Interaction Harms—not

just toxicity or explicit policy violations—could help close the gap in areas where safety finetuning remains less effective. Third, extending the taxonomy and orchestration layer to multilingual and multimodal settings would enable evaluation of cross-lingual and cross-modal jailbreaks, including harms specific to images, audio, and code.

Finally, integrating this logging backend with interactive visual analytics—building on ideas from AdversaFlow—could provide practitioners with both quantitative dashboards and qualitative drill-down tools on top of the same persistent data substrate. Beyond research prototypes, deploying the pipeline in operational settings will require addressing engineering and governance questions such as cost, latency, access control, and coordinated disclosure of discovered vulnerabilities. Addressing these challenges would move the field closer to standardized, reproducible red-teaming infrastructure for large language models.

References

- [1] Dazhen Deng, Chuhan Zhang, Huawei Zheng, Yuwen Pu, Shouling Ji, and Yingcai Wu. 2025. AdversaFlow: Visual Red Teaming for Large Language Models with Multi-Level Adversarial Flow. *IEEE Transactions on Visualization and Computer Graphics* 31, 1 (2025). doi:10.1109/TVCG.2024.3456150 Early access 2024.
- [2] Emily Dinan, Samuel Humeau, Bharath Chintagunta, and Jason Weston. 2019. Build It Break It Fix It for Dialogue Safety: Robustness from Adversarial Human Attack. In *Proceedings of EMNLP-IJCNLP*. doi:10.18653/v1/D19-1461
- [3] Deep Ganguli, Liane Lovitt, Jonathan Kernion, Amanda Askell, Yuntao Bai, et al. 2022. Red Teaming Language Models to Reduce Harms: Methods, Scaling Behaviors, and Lessons Learned. arXiv:2209.07858
- [4] Song Ge, Chunting Zhou, Rui Hou, Madian Khabza, Yu-Chen Wang, Qifan Wang, Jiawei Han, and Yuxiang Mao. 2023. MART: Improving LLM Safety with Multi-round Automatic Red-Teaming. arXiv:2311.07674
- [5] Zihan Lin, Zening Wang, Yuxuan Tong, Yuxuan Wang, Yuchen Guo, Yisen Wang, and Jingbo Shang. 2023. ToxicChat: Unveiling Hidden Challenges of Toxicity Detection in Real-World User-AI Conversation. In *Findings of EMNLP*. doi:10.18653/v1/2023.findings-emnlp.311
- [6] OpenAI. 2024. GPT-4 Technical Report. arXiv:2303.08774
- [7] Ethan Perez, Sam Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. 2022. Red Teaming Language Models with Language Models. In *Proceedings of EMNLP*. doi:10.18653/v1/2022.emnlp-main.225
- [8] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of ACL*. doi:10.18653/v1/2020.acl-main.442
- [9] Mark Russinovich et al. 2024. PyRIT: A Framework for Security Risk Identification and Red Teaming in Generative AI Systems. In *Proceedings of the 2024 Microsoft AI Red Teaming Workshop*. arXiv:2410.02828.
- [10] L. Sun, Y. Huang, H. Wang, S. Wu, Q. Zhang, et al. 2024. TrustLLM: Trustworthiness in Large Language Models. arXiv:2401.05561
- [11] Rohan Taori et al. 2023. Alpaca: A Strong, Replicable Instruction-Following Model. arXiv:2303.16199 Dataset and SFT recipe.
- [12] Yuxin Wang, Hongyin Li, Xu Han, Preslav Nakov, and Timothy Baldwin. 2024. Do-Not-Answer: A Dataset for Evaluating Safeguards in LLMs. In *Findings of EACL*.
- [13] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How Does LLM Safety Training Fail?. In *Proceedings of NeurIPS*.
- [14] Laura Weidinger, John Mellor, et al. 2021. Ethical and Social Risks of Harm from Language Models. arXiv:2112.04359
- [15] Laura Weidinger, Jonathan Uesato, et al. 2022. Taxonomy of Risks Posed by Language Models. In *Proceedings of ACM FAccT*. doi:10.1145/3531146.3533088
- [16] Daniel Ziegler, Sam Nix, Leo Chan, Tom Bauman, Peter Schmidt-Nielsen, Tony Lin, Adam Scherlis, Naoto Nabeshima, Ben Weinstein-Raun, Daniel de Haas, Beth Shlegeris, and Nate Thomas. 2022. Adversarial Training for High-Stakes Reliability. In *Proceedings of NeurIPS*.